

VPS: Rethinking OLTP Database Performance Evaluation through Transactional Value

Jianbin Qin, Yibin Lin, Wendi Hua, Rui Mao✉

Shenzhen Institute of Computing Sciences

Shenzhen University

Shenzhen, China

{qinjianbin, mao}@szu.edu.cn, linyibin1234@126.com, huawendi2023@email.szu.edu.cn

Chuan Xiao

Osaka University

Nagoya University

Osaka, Japan

chuanx@ist.osaka-u.ac.jp

Abstract—In this paper, we propose Value per Second (VPS), a novel performance metric for OLTP databases that incorporates transactional value into the performance evaluation framework, addressing the weaknesses of traditional throughput- and latency-based metrics. VPS provides a more accurate reflection of real-world business performance. We extend the TPC-C benchmark as an example to illustrate the evaluation insights enabled by VPS and highlight the challenges in achieving high VPS in database systems.

In summary, we envision that the next generation of database evaluation should integrate transaction value with system throughput. VPS indisputably better aligns with business objectives and operational significance, offering a more meaningful performance metric for end users. Currently, VPS is in the early stage of development. Several open challenges remain, including cross-domain value modeling, value prediction, and the design of value-aware transaction scheduling algorithms optimized for VPS. We encourage collaboration among researchers across communities to advance this emerging direction.

Index Terms—database performance, Value Per Second, benchmark, transaction scheduling

I. INTRODUCTION

Online transaction processing (OLTP) databases are critical to modern enterprise applications [1]–[3]. However, under certain workload conditions—such as flash sale events on e-commerce platforms, where millions of purchase transactions flood the system within seconds—databases often fail to meet stringent performance requirements. Traditionally, this problem has been addressed by scaling database resources either vertically or horizontally. Yet, both approaches introduce significant drawbacks: scaling increases infrastructure costs and may compromise fault tolerance and data consistency [4]. We observe that, in modern enterprise applications, transactions inherently possess value attributes — that is, different transactions contribute unequally to the overall business profit once completed. Therefore, when the database cannot meet the performance requirements under specific workload conditions, optimizing the scheduling strategy to prioritize high-value transactions — even at the cost of increased latency for low-value ones — can help maximize overall business profit. This approach not only avoids the drawbacks associated with database scaling but also enhances the database’s value throughput.

Running Example. In a high-concurrency e-commerce “flash sale” scenario receiving millions of orders ranging from low-value accessories (\$39) to high-value systems (\$20,000). A throughput-oriented scheduler may process numerous low-value orders rapidly while delaying high-value ones, potentially causing revenue loss through customer abandonment. In contrast, a value-aware scheduler prioritizes high-value transactions, marginally reducing overall throughput but significantly improving total business profit—a critical metric in enterprise operations.

At present, the performance of OLTP systems is typically evaluated using benchmarks such as TPC-C [5], TPC-E [6], Sysbench [7], and SmallBank [8]. Among these, TPC-C, TPC-E, and SmallBank primarily simulate specific business scenarios and assess performance based on database throughput (transactions per second, TPS), whereas Sysbench, in addition to measuring throughput, also monitors CPU utilization and memory usage. Despite these efforts, existing benchmarks remain largely throughput-centric and provide no effective means to evaluate value-oriented database scheduling schemes. This limitation highlights the need for new performance metrics that explicitly capture the value dimension of transactional workloads and facilitate the development of value-driven scheduling algorithms. To address this issue, we propose VPS (Value per Second), a novel metric that evaluates database scheduling algorithms from a value-driven perspective, thus fostering the development of value-oriented scheduling algorithms.

Unlike traditional throughput-based metrics, VPS explicitly incorporates the notion of transactional value, enabling a unified evaluation of both system efficiency and business benefit. To capture the throughput–profit trade-off explicitly, we define Value Per Second (VPS) as the total transactional value completed per second.

Based on this metric, we design and implement a novel Benchmark. This Benchmark is built upon the e-commerce retail scenario of the TPC-C, where the behaviors of both users and database administrators are abstracted into five representative transaction types, each assigned a systematic value characterization to ensure comparability across different transaction classes. To better approximate real-world conditions, we further incorporate a random termination mechanism

that simulates user cancellations caused by latency. In addition, the Benchmark supports multiple product price distributions (e.g., Pareto, uniform, and Gaussian distributions), enabling researchers to flexibly adjust experimental conditions and explore how variations in transaction values influence database scheduling strategies. Finally, we conduct systematic experiments on two mainstream database systems, PostgreSQL and MySQL. The results demonstrate that the proposed Benchmark is not only practical and portable but also capable of effectively capturing differences in scheduling strategies from a value-oriented perspective, thus providing new tools and evaluation methodologies for database performance research.

The main contributions of this paper are as follows:

- We propose the database performance metric of VPS for the first time, providing a value-oriented measure of database performance that captures the total transaction value completed per unit of time.
- We design and implement a VPS-centric database Benchmark extended on TPC-C that abstracts five representative transaction types in an e-commerce context, each with a systematic value definition.
- We incorporate a random termination mechanism to simulate user behavior under latency and support multiple price distribution models, thereby making the evaluation environment closer to real-world business scenarios.

The rest of the paper is organized as follows. Section II discusses related work. Section III presents the VPS version of the TPC-C benchmark design, including schema, transactions, value definitions, and user behavior modeling. Experiments are reported in Section IV. Section V concludes the paper.

II. RELATED WORK

A. Early Studies on Value-based Scheduling

Value-based scheduling has been studied since three decades ago. Haritsa et al. [9] explored the relationship between transaction value and scheduling strategies in real-time databases. Their work focused on real-time systems with explicit transaction deadlines, which differ fundamentally from OLTP databases where transactions have no clear deadlines. Pang et al. [10] proposed the PMM algorithm for real-time databases, which allocates more memory resources to higher-priority queries. Transaction priority in their work is defined based on deadlines rather than business value. Davison and Graefe [11] introduced economic principles into database resource scheduling by treating system resources as currency and optimizing resource allocation during query execution. In contrast, our work focuses on value-oriented transaction scheduling, where execution priority is determined by predicting transaction value upon arrival rather than reallocating resources during execution.

B. Database Benchmarks

Database benchmarking has evolved since 1980s [12], [13], with the goal of simulating real-world workloads to evaluate

system performance [14], [15]. TPC-C [5] and TPC-E [6] are widely used OLTP benchmarks that model wholesale and brokerage applications, respectively. YCSB [16] targets key-value stores, while SmallBank [8] and DebitCredit [17] simulate banking applications. CH-benCHmark [18] (also known as TPC-CH) integrates TPC-C and TPC-H [19] for Hybrid Transactional/Analytical Processing (HTAP) systems [20], [21]. More recently, PeakBench [22] is proposed to capture high concurrency and contention e-commerce flash-sale scenarios.

However, none of these benchmarks incorporate transaction value as a first-class metric. They focus on throughput, latency, and scalability, but do not reflect the economic efficiency of database systems. This highlights the need for a novel benchmark that incorporates the value dimension in evaluating database performance.

C. User Behavior and Latency

User behavior under latency has been studied extensively. Card et al. [23] defined the “instantaneous response constant” of one second. It is a critical threshold for maintaining the perception of smooth interaction and uninterrupted cognitive processes. When this limitation is beyond, additional feedback is required. Building upon this, Nielsen [24] summarized three key response time thresholds: delays below 0.1 seconds are virtually imperceptible, delays around one second are noticeable, and delays of ten seconds approach the limit of sustained user attention. In 2015, Google’s RAIL model [25] recommends 100ms for user input feedback in the web applications and interactive systems.

Subsequent empirical studies show that latency directly impacts business outcomes. Web pages [8] and electronic retails [26] are reported significant revenue loss with increased latency. Specifically, Santiago et al. [27] find that sales drop by 4.2% per 10% latency increase. Amazon internal analysis [28] shows that every additional 100ms response time leads to an 1% sale decrease, while other work [29] indicates that even 1s delay can reduce user satisfaction by 16%.

Despite these findings, no benchmarks systematically integrates user abort behavior into transaction evaluation. Our work bridges this gap by modeling user abort probabilities and value loss due to latency.

III. BENCHMARK FOR VALUE PER SECOND

We propose a transaction-value-oriented database benchmark, an extension of TPC-C that incorporates transaction value and user behavior modeling. The benchmark evaluates both traditional metrics (TPS, latency) and value per second (VPS). For traditional metrics, 50% transactions’ latency (p_{50}), 99% transactions’ latency (p_{99}), and 99.9% transaction’s latency (p_{999}) are evaluated. VPS captures the actual business value accomplished per unit time, offering a closer reflection of real-world application requirements. Therefore, the transaction-value-oriented database benchmark provides a more comprehensive evaluation of OLTP performance.

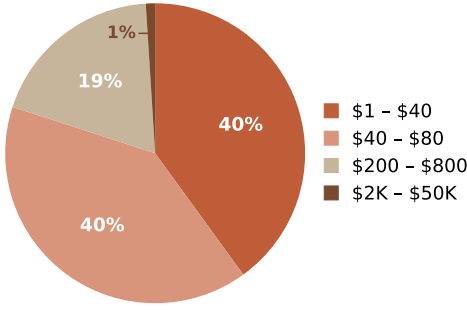


Fig. 1: Item Value Distribution in TPC-C.

A. Schema

We adopted the table schema of TPC-C with modifications to better reflect e-commerce retail scenarios. The database consists of nine tables: WAREHOUSE, DISTRICT, STOCK, ITEM, ORDERLINE, ORDER, NEWORDER, CUSTOMER, and HISTORY. To simulate wholesale business operations, our benchmark models an e-commerce setting. We added an *isPay* attribute to the NEWORDER table to indicate whether an order has been paid.

We further improve the database initialization, particularly in the construction of the ITEM table. In real-world e-commerce, order values often follow the Pareto principle [30]. Since the value of an order depends on the items purchased, and random generation cannot accurately reflect realistic value distributions, we control the distribution indirectly by configuring item prices. Three types of value distributions are supported: Pareto-based, uniform, and Gaussian. In the Pareto-based distribution, following Kakalejvcik et al. [31], items are assigned prices as follows: 1% of items are priced between \$2,000 and \$50,000, 19% between \$200 and \$800, 40% between \$40 and \$80, and 40% between \$1 and \$40. The top 1% of items are intentionally set as extremely high-value to simulate the head-effect observed in real markets. Fig. 1 shows this distribution. During the initialization of the NEW-ORDER and ORDER-LINE tables, order values are computed as the sum of included items rather than being randomly generated. Although these adjustments increase initialization time, they significantly enhance the benchmark’s representativeness and fidelity to real e-commerce scenarios.

B. Transaction

In designing Benchmark, we adopted an e-commerce scenario as the background and incorporated typical workflows of both users and merchants. Based on this setting, we defined five core transaction types: *New-Order*, *Payment*, *Stock-Level*, *Order-Status*, and *Delivery*. These transactions collectively capture the essential interactions in e-commerce applications, among which *New-Order* and *Payment* are the most common and frequent operations. It is worth noting that users exhibit varying levels of tolerance to latency across different transaction types. Moreover, these transactions differ in their access

TABLE I: Transaction Read/Write Types and Tolerance

Transaction Type	Access Pattern	Delay Tolerate
<i>New-Order</i>	Read-Write	Low
<i>Payment</i>	Read-Write	Low
<i>Stock-Level</i>	Read-Write	Moderate
<i>Order-Status</i>	Read-Only	High
<i>Delivery</i>	Read-Write	High

patterns. A detailed classification of the five transaction types is provided in Table I.

1) *New-Order*: The *New-Order* transaction is responsible for order creation. The transaction randomly selects 1~10 items from the ITEM table, with each item assigned a purchase quantity between 1 and 15. Once the order is generated, its total value is computed, and the corresponding information is updated into the NEW-ORDER, ORDER-LINE, DISTRICT, STOCK, CUSTOMER, and ORDER tables. It is noteworthy that if any item in the order has insufficient stock, the current transaction is immediately aborted, and a new *Stock-Level* transaction is triggered to replenish the corresponding inventory. Meanwhile, during order generation, the system maintains a global hot items table using the least recently used (LRU) algorithm, which serves as a reference for value calculation in subsequent *Stock-Level* transactions.

2) *Payment*: The *Payment* transaction is responsible for completing order payments. It randomly selects an order from the NEWORDER table that has been submitted but not yet paid (i.e., the attribute *isPay* is set to 0). The transaction then updates the *isPay* to 1 to indicate that the order has been paid. In addition, the CUSTOMER table is updated to reflect changes in the customer’s account information, while the WAREHOUSE, DISTRICT, and HISTORY are also modified to record the payment and its effects on related entities.

3) *Stock-Level*: The *Stock-Level* transaction is responsible for replenishing items with insufficient inventory. In the normal case, this transaction scans the most recent 20 orders within a given district to identify the items involved. Based on this information, it retrieves the 50 items with the lowest inventory levels from the STOCK table and replenishes each item by adding 2000 units to its stock. Moreover, when a *New-Order* transaction detects insufficient inventory during the order creation process, it immediately triggers a *Stock-Level* transaction. In this scenario, the *Stock-Level* transaction directly receives the set of items reported as understocked by the *New-Order* transaction, thereby bypassing the scan of the DISTRICT and STOCK and proceeding to update the STOCK directly.

4) *Delivery*: The *Delivery* transaction is responsible for delivering customer orders. It retrieves the 10 earliest submitted orders from the *New-Order* table that have already been paid. The *Delivery* transaction then removes these 10 orders from the *New-Order* and updates the related records in the CUSTOMER, ORDER, and ORDERLINE accordingly to complete the delivery process.

5) *Order-Status*: The *Order-Status* transaction is responsible for retrieving customer order information. This transaction supports two query modes: (1) using the customer's last name to retrieve all associated order identifiers, or (2) directly specifying the unique customer identifier (c_id) to locate the customer's orders. Once the order identifiers are obtained, the system retrieves the corresponding details from the ORDERLINE table, including product types, quantities, and prices.

C. Transaction Value

In different scenarios, the specific definition of transaction value may vary. However, the underlying methodology remains consistent. The process begins with identifying the core business object of the scenario, followed by weighing transactions according to their latency tolerance and the operations performed on the core object, thereby determining the overall transaction value. In the TPC-C scenario, the order is the core business object, and thus the definition of transaction value can be constructed around orders. In our proposed VPS-Benchmark, the order is also regarded as the core business object. Based on this scenario and the transaction logic defined in the benchmark, we introduce a general Eq. (1) to provide a unified definition of transaction value across different transaction types.

$$V_{trx} = \alpha \cdot p_o + \beta \cdot p_h \quad (1)$$

where p_o is the current order value, p_h is the historical average order value for the customer, and α, β are type-specific weights.

When specifying transaction value definitions for different transaction types, variations naturally arise due to differences in their execution logic and the underlying tables involved. In our model, the potential value of users is taken into account, which may affect a company's future revenue. To capture this, we incorporate the historical order value of a customer as a proxy for the user's potential value. For instance, in the case of *New-Order* transactions, the value definition additionally considers the historical order value associated with the corresponding customer. By integrating such information, we establish a more comprehensive value definition model that facilitates cross-transaction comparison. The parameters of weight α and β are adjusted according to the transaction type. The specific definitions of transaction value for each type in the VPS-Benchmark are given as follows:

$$\begin{aligned} V_{New-Order} &= 0.7 \cdot p_o + 0.3 \cdot p_h \\ V_{Payment} &= 0.9 \cdot p_o \\ V_{Stock-Level} &= \begin{cases} 0.8 \cdot p_{vo}, & \text{if no hot items exist} \\ 1.0 \cdot p_{vo}, & \text{if hot items exist} \end{cases} \\ V_{Delivery} &= 0.5 \cdot p_o \\ V_{Order-Status} &= 0.2 \cdot p_o \end{aligned} \quad (2)$$

In Eq. (2), $V_{New-Order}$ defines the value of a *New-Order* transaction. p_o represents the value of the order currently being processed, and p_h denotes the average value of the historical orders associated with the customer of this order. By adjusting α and β , the historical order value is incorporated because it often reflects the customer's potential purchasing power. Prioritizing customers with higher potential can help increase business revenue.

$V_{Payment}$ defines the value of a *Payment* transaction, where p_o is the value of the current order. Since users typically have a higher delay tolerance for *Payment* compared to *New-Order* transactions, α is set to 0.9 to slightly lower the transaction priority. This transaction generates immediate revenue, so potential future value is not considered.

$V_{Stock-Level}$ defines the value of a *Stock-Level* transaction. Considering its transaction logic, a virtual order of "50 units per item" is created for each item that needs restocking, and the average value of these virtual orders is taken as the value of the core business order. Two scenarios are distinguished, reflecting the transaction's importance under different conditions. If no hot items exist, α is set to 0.8, and α is set to 1.0 when hot items exist.

$V_{Delivery}$ defines the value of a *Delivery* transaction. The average value of the orders to be processed is taken, and considering the higher delay tolerance of *Delivery*, α is set to 0.5. $V_{Order-Status}$ defines the value of an *Order-Status* transaction. This is a read-only transaction, and the average value of the queried orders is taken. As it has the highest delay tolerance and minimal impact on user revenue, α is set to 0.2.

D. User Abort

In real-world application scenarios, system latency can significantly affect user experience, often leading to premature termination of transactions and consequently reducing merchants' profits. However, most existing benchmarks [5]–[8] fail to capture or simulate this phenomenon. To address this gap, our proposed benchmark incorporates a mechanism that models user-initiated transaction aborts caused by latency. Specifically, when transaction latency exceeds a certain threshold, users do not abort immediately. Instead, there exists a probability of abort that gradually increases with latency until it ultimately reaches 100%.

Prior studies [23]–[25] show that delays shorter than 100ms are generally perceived as instantaneous responses and thus imperceptible to users, whereas delays longer than 1s typically require additional system feedback to maintain user engagement. Based on these findings, we define 100ms as the threshold at which the abort rate begins to increase. Once latency exceeds 100ms, the abort probability grows continuously with increasing delay. When latency reaches 1s, the transaction is aborted outright.

To realize this mechanism, we design an independent timer for each transaction and define that the abort rate increases with every 10% increment in latency. During transaction

execution, after each database call is completed, the system updates the timer's timestamp and recalculates the current abort probability. Since different transaction types exhibit varying sensitivities to latency, we incorporate a transaction-specific weighting factor into the calculation to reflect these differences in real-world scenarios. Based on the computed probability, the benchmark randomly aborts transactions, thereby providing a realistic simulation of user behavior where excessive waiting times trigger transaction abandonment. The abort probability is formally defined in Eq. (3).

$$P_{Abort} = \begin{cases} 0, & d \leq 0.1s \\ \min(1, \frac{d-100}{10\%*100} * p_{type}), & 1s \geq d > 0.1s \\ 1, & d > 1s \end{cases} \quad (3)$$

where d denotes the current latency and p_{type} represents the abort rate associated with the corresponding transaction type.

E. Transaction Value Loss

Due to the existence of user-initiated aborts, a transaction that is terminated prematurely not only fails to complete but also results in a loss of its associated value. For transaction scheduling algorithms, accurately estimating potential value loss is therefore a critical issue. Building upon the abort mechanism introduced earlier, we propose a transaction value loss model to quantify the degradation in value caused by latency-induced aborts. With this model, the scheduler can prioritize transactions whose termination would lead to substantial losses, thereby improving overall system profitability. The formulation of the loss function is grounded in the abort probability. As the abort probability increases, the expected value of a transaction gradually diminishes. Eq. (4) characterize the value loss of transactions.

$$Loss = \begin{cases} 0, & d \leq \bar{d} \\ V_{trx} * \frac{d-\bar{d}}{10\%*\bar{d}} * w_{type}, & d > \bar{d} \end{cases} \quad (4)$$

where $\bar{d}$ denotes the delay threshold. Considering that human perception of delay is approximately $327.2 \text{ ms} \pm 89.7 \text{ ms}$, we set the threshold to $\bar{d} = 200 \text{ ms}$. When the transaction delay d exceeds this threshold, users can perceive the delay and respond accordingly. V_{trx} represents the value of the transaction, and w_{type} reflects users' sensitivity to the delay for different types of transactions. A larger w_{type} indicates lower tolerance to delay and a higher value loss consequently. We assign the following w_{type} values for different transaction types: $w_{New-Order} = 0.8\%$, $w_{Payment} = 1.7\%$, $w_{Stock-Level} = 0.7\%$, $w_{Delivery} = 0.5\%$, and $w_{Order-Status} = 0.3\%$. By defining the loss function in this way, the transaction value dynamically changes over time, thereby more accurately simulating real-world environments.

F. Heuristic Scheduling Algorithm

We design a simple heuristic scheduling algorithm that assumes transaction values are known upon arrival without additional computation. Transactions are inserted into a value-priority heap and executed in order of decreasing value.

While value-driven, this approach requires transaction values as inputs, which are often unavailable or inaccurate in online settings. We evaluate this algorithm experimentally. Note that the issue of optimal scheduling in online scenarios is beyond the scope of this paper.

IV. EXPERIMENTS

We ran our experiments on a server with 2 AMD EPYC 7543 CPUs, each with 32 cores and 2 hardware threads per core, and 512 GB of main memory in total. Considering that modern database systems are widely deployed in cloud environments and typically operate on virtual or containerized infrastructures, we deployed PostgreSQL 15.13 [32] and MySQL 8.0.40 [33] within Docker containers for experimental evaluation to better reflect practical deployment scenarios. This study focuses on the relative performance differences among scheduling strategies rather than absolute throughput. All experiments were conducted under identical hardware configurations and within a unified container environment to ensure comparability. Consequently, any systematic overhead introduced by containerization does not affect the validity of our observations. PostgreSQL was configured with 2 GB of shared buffers, and synchronous commit was enabled. The evaluation metrics include transactions per second (TPS), value per second (VPS), as well as the overall performance of the databases under different scheduling strategies.

In our experiments, we generate a set of transactions using a benchmark and compute their values according to the definition in Section III. Each transaction is annotated with its arrival time to construct an offline workload. We assume that all transactions can be executed without deadlocks, and compute the theoretical optimal VPS under a value-priority policy, which serves as the baseline.

In this offline workload, we emulate database stress conditions by regulating the number of transactions arriving within a fixed time window. Under the configuration of 16 warehouses, we evaluate system performance by controlling the transaction arrival rate to mimic a scenario where 128 transactions arrive every 300 ms. To further examine how concurrency affects system behavior, we vary the number of worker threads to adjust the degree of parallelism. We systematically analyze the variations in VPS and throughput under different levels of concurrency.

Fig. 2a shows the trend of throughput variation with increasing terminals under the warehouse = 16 configuration for both MySQL and PostgreSQL. We observe that, after introducing the priority queue mechanism, the throughput of MySQL remains relatively stable, whereas in PostgreSQL, the throughput reaches its peak at a later stage. We attribute this difference to the distinct internal transaction scheduling strategies adopted by MySQL and PostgreSQL. Moreover, since PostgreSQL inherently exhibits lower throughput, the additional overhead introduced by the priority queue layer—where transactions must be enqueued before entering the database—further delays transaction processing and slightly degrades overall performance. Fig. 2c and 2d illustrate the variation of VPS with in-

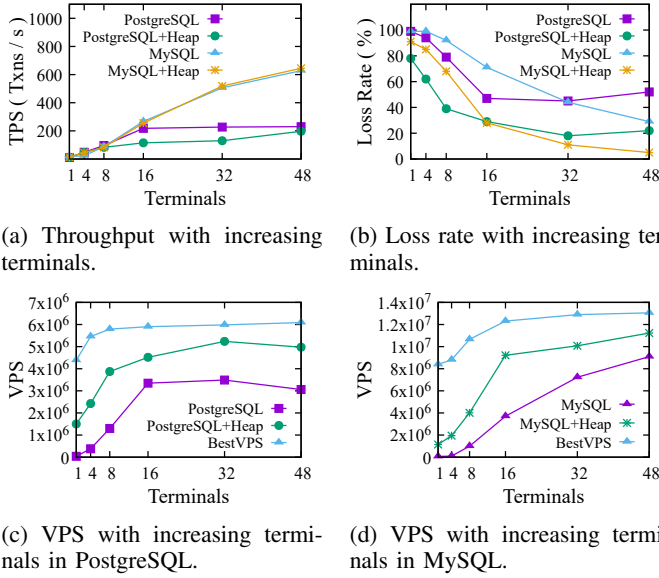


Fig. 2: Database Performance with increasing terminals.

creasing terminals under the warehouse = 16 configuration for PostgreSQL and MySQL, respectively. Across both systems, we observe that introducing the priority queue mechanism leads to a noticeable improvement in VPS approximately 50% higher than the original baseline. Besides, we computed the theoretical upper bound of VPS based on the database throughput without the priority queue. This provides an idealized reference for evaluating the performance ceiling of each system. By reducing the number of terminals, we simulate resource-constrained database environments. Our analysis reveals that as the number of terminals decreases, the gap between the optimal VPS and the observed VPS widens. This is mainly because, as the number of terminals decreases, the system becomes unable to sustain the required transaction throughput under the given workload, thereby forcing the database to drop or defer certain transactions. Therefore, the transaction dropping policy has a significant impact on the database's VPS, as it directly determines which transactions are preserved or discarded under constrained throughput conditions. When the priority queue is introduced, the system preferentially drops low-value transactions, thereby reducing the value loss and narrowing the gap with the theoretical optimum.

When resources are sufficient and the system throughput meets workload demand, the impact of different dropping policies on VPS becomes marginal. Fig. 2b illustrates the value proportion of dropped transactions. The results show that the value-aware scheduling mechanism can significantly mitigate the loss caused by interrupted transactions under the same resource constraints. These findings validate the effectiveness of the proposed benchmark design and confirm our earlier observation on the relationship between VPS and the number of terminals, highlighting the potential of value-driven scheduling in enhancing system efficiency.

Our benchmark supports three configurable item price distributions: the Pareto distribution, the uniform distribution, and the Gaussian distribution. In the following experiments, we focus on analyzing the impact of the standard deviation of transaction values on scheduling performance. To this end, we set the item price distribution to uniform and control the standard deviation of transaction values by adjusting the range of item prices. Experimental results on the MySQL database (see Table II) demonstrate that the standard deviation of transaction values can indeed be effectively controlled by modifying the item price range. As illustrated in Fig. 3, when the item price range increases, the standard deviation of transaction values rises accordingly, and the difference in value loss rate between the value-aware scheduling algorithm and the baseline scheduling method also becomes more pronounced.

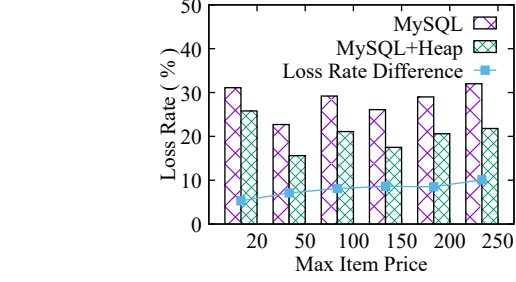


Fig. 3: Value loss rate with increasing range of item price

TABLE II: Value loss rate in different item range

Range	SD	Original Loss Rate	Heap Loss Rate	Difference
10~20	140.81	31.07%	25.76%	5.31%
10~50	292.23	22.68%	15.62%	7.06%
10~100	551.93	29.22%	21.10%	8.12%
10~150	778.67	26.10%	17.53%	8.57%
10~200	1052.16	29.03%	20.57%	8.46%
10~250	1399.39	31.96%	21.85%	10.11%

These observations indicate that when the standard deviation of transaction values is small, the performance improvement achieved by the value-aware scheduling algorithm is limited. In contrast, when the standard deviation is large, the algorithm yields significant performance gains. The underlying reason is that in high-variance scenarios, certain high-value transactions may have a value equivalent to the aggregate value of many low-value transactions. Prioritizing such high-value transactions thus compensates for the value loss caused by the time-out of low-value transactions. In summary, the value-aware scheduling algorithm provides greater performance benefits when the variance of transaction values is high. Determining the conditions under which this algorithm should be activated to maximize benefits remains an open research problem.

V. DISCUSSIONS AND RESEARCH DIRECTIONS

Based on our experimental results, we observe that there remain many open questions in the field of VPS.

A. Research Direction #1: Towards Optimal Value-Oriented Transaction Scheduling

We initially designed a simple algorithm to schedule transactions according to their transactional value. However, we found that this algorithm exhibits certain limitations: while it prioritizes high-value transactions, it can lead to a reduction in overall database throughput. Moreover, under resource-constrained scenarios, the performance of the proposed algorithm still deviates significantly from the theoretical optimal VPS. Specifically, in both PostgreSQL and MySQL, when the number of concurrent threads ranges from 1 to 8, the VPS achieved by our algorithm can be up to twice lower than the theoretical optimum. This indicates that further research on value-oriented scheduling algorithms is necessary. Future work should focus on developing algorithms capable of approaching the theoretical optimal VPS while minimizing any negative impact on the database's inherent performance.

B. Research Direction #2: Timing and Fairness of Value-Oriented Transaction Scheduling

We conducted experiments to evaluate our proposed simple algorithm under different transaction value distributions. Our observations indicate that as the variance of transaction values increases, the gap between the VPS achieved by the value-oriented scheduling scheme and that of the original scheduling scheme also grows. Moreover, value-oriented scheduling does not inherently guarantee fairness. When fairness is considered, it is not desirable to apply a purely value-oriented scheduling scheme at all times. Thus, determining the appropriate conditions for enabling value-oriented scheduling is critical. Exploring the timing and triggers for activating value-oriented scheduling can help maintain fairness while maximizing overall profit in high-load and high-concurrency scenarios.

C. Research Direction #3: Towards a Cross-Domain Transaction Value-Definition and VPS Benchmark for OLTP Systems

The value definition adopted in this work is tailored to a specific business scenario and therefore has inherent limitations. In particular, the current model does not explicitly capture customers' long-term potential value, nor does it systematically quantify the potential loss in user experience and long-term revenue caused by delaying or aborting low-value transactions.

Nevertheless, the value function can be extended and refined according to practical business objectives. In real-world deployments, factors such as customer lifetime value, user loyalty, and historical behavioral characteristics may be incorporated to balance short-term revenue optimization with long-term customer relationship management.

Furthermore, OLTP systems are widely used across diverse domains, including finance, e-commerce, and logistics, each characterized by distinct business semantics and value definitions. This domain heterogeneity limits the general applicability of the proposed benchmark. Future work should explore domain-aware value modeling and aim to develop a more

general framework capable of supporting transaction value definitions across different application contexts.

D. Research Direction #4: Online Transaction Value Prediction for Value-Oriented OLTP Scheduling

The experiments conducted in this work are currently performed in an offline setting, where transaction values are known in advance. In real-world scenarios, however, transaction values are typically unknown beforehand. Therefore, it is necessary to predict transaction values during the database's scheduling process. Accurate prediction requires not only the SQL statements of the transactions but also domain-specific knowledge. Considering that OLTP databases generally have low tolerance for transaction delays, the scheduling algorithm must predict transaction values and make scheduling decisions within a short time frame. Developing methods for online prediction of transaction values—both within a single domain and across multiple domains—will significantly support the advancement of value-oriented transaction scheduling algorithms.

VI. CONCLUSION

This study introduces a novel database performance metric, value per second (VPS), and develops a comprehensive benchmark tailored for VPS evaluation. The benchmark primarily simulates user interactions with databases in e-commerce scenarios, encompassing not only standard transactional operations but also cases where users abort transactions due to latency. Moreover, we refine the transaction logic to better reflect real-world application requirements. Building upon this benchmark, we propose a heuristic value-aware scheduling algorithm that prioritizes high-value transactions, thereby improving overall database performance.

Although the effectiveness of value-aware scheduling has been demonstrated, several open research challenges remain. For instance, enabling databases to automatically infer transaction values, designing more efficient value-aware scheduling algorithms, and identifying the application scenarios in which such strategies yield the greatest benefits are important directions for future research.

REFERENCES

- [1] M. Stonebraker *et al.*, "Enterprise database applications and the cloud: Emerging patterns and problems," in *Proceedings of IC2E (International Conference on Cloud Engineering?)*, 2014, available online. [Online]. Available: <https://www.cs.cmu.edu/~pavlo/papers/stonebraker-ic2e2014.pdf>
- [2] F. Ozcan, Y. Tian, and P. Tözün, "Hybrid transactional/analytical processing: A survey," in *Proceedings of the 2017 ACM International Conference on Management of Data*, 2017, pp. 1771–1775.
- [3] S. Harizopoulos, D. J. Abadi, S. Madden, and M. Stonebraker, "Oltp through the looking glass, and what we found there," in *Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker*, 2018, pp. 409–439.
- [4] U. K. Manne, "Horizontal vs. vertical scaling in modern database systems: A comparative analysis of performance and cost trade-offs," *International Journal of Computer Engineering and Technology (IJCET)*, vol. 15, no. 5, pp. 514–524, 2024, accessed: 2025-11-07. [Online]. Available: https://iaeme.com/Home/article_id/IJCET_15_05_048

- [5] S. T. Leutenegger and D. Dias, "A modeling study of the tpc-c benchmark," *ACM Sigmod Record*, vol. 22, no. 2, pp. 22–31, 1993.
- [6] T. P. P. C. TPC, "Tpc benchmark™ e," 2010.
- [7] P. Zaitsev and V. Tkachenko, "Sysbench: A scriptable multi-threaded benchmark tool," <https://github.com/akopytov/sysbench> 2007, accessed: 2025-08-11.
- [8] J. Brutlag, "Speed matters for google web search," *Google. June*, vol. 2, no. 9, 2009.
- [9] J. R. Haritsa, M. J. Carey, and M. Livny, "Value-based scheduling in real-time database systems," *VLDB J.*, vol. 2, no. 2, pp. 117–152, 1993. [Online]. Available: <http://www.vldb.org/journal/VLDBJ2/P117.pdf>
- [10] H. H. Pang, M. J. Carey, and M. Livny, "Managing memory for real-time queries," *ACM SIGMOD Record*, vol. 23, no. 2, pp. 221–232, 1994.
- [11] D. L. Davison and G. Graefe, "Dynamic resource brokering for multi-user query execution," in *Proceedings of the 1995 ACM SIGMOD international conference on Management of data*, 1995, pp. 281–292.
- [12] J. Gray, *Benchmark handbook: for database and transaction processing systems*. Morgan Kaufmann Publishers Inc., 1992.
- [13] D. Bitton, D. J. DeWitt, and C. Turbyfill, *Benchmarking database systems: A systematic approach*. Computer Sciences Department, University of Wisconsin-Madison, 1983.
- [14] V. Persico, A. Pescapé, A. Picariello, and G. Sperli, "Benchmarking big data architectures for social networks data processing using public cloud platforms," *Future Generation Computer Systems*, vol. 89, pp. 98–109, 2018.
- [15] Y. Tay, "Data generation for application-specific benchmarking," *Proceedings of the VLDB Endowment*, vol. 4, no. 12, pp. 1470–1473, 2011.
- [16] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with ycsb," in *Proceedings of the 1st ACM symposium on Cloud computing*, 2010, pp. 143–154.
- [17] M. Stonebraker, "A measure of transaction processing power," in *Readings in database systems (2nd ed.)*, 1994, pp. 442–454.
- [18] R. Cole, F. Funke, L. Giakoumakis, W. Guy, A. Kemper, S. Krompass, H. Kuno, R. Nambiar, T. Neumann, M. Poess *et al.*, "The mixed workload ch-benchmark," in *Proceedings of the Fourth International Workshop on Testing Database Systems*, 2011, pp. 1–6.
- [19] T. P. P. Council, "Tpc-h benchmark specification," *Published at http://www.tpc.org/hspec.html*, vol. 21, pp. 592–603, 2008.
- [20] C. Zhang, G. Li, J. Zhang, X. Zhang, and J. Feng, "Htap databases: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 11, pp. 6410–6429, 2024.
- [21] G. Li and C. Zhang, "Htap databases: What is new and what is next," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 2483–2488.
- [22] C. Zhang, Y. Li, R. Zhang, W. Qian, and A. Zhou, "Benchmarking on intensive transaction processing," *Frontiers of Computer Science*, vol. 14, no. 5, p. 145204, 2020.
- [23] S. K. Card, G. G. Robertson, and J. D. Mackinlay, "The information visualizer, an information workspace," in *Proceedings of the SIGCHI Conference on Human factors in computing systems*, 1991, pp. 181–186.
- [24] J. Nielsen, "Response times: the three important limits," *Usability Engineering*, 1993.
- [25] P. Irish and P. Lewis, "Introducing rail: A user-centric model for performance," *Smashing Magazine*, October, vol. 2, 2015.
- [26] G. B. Yom-Tov, L. Yedidsion, and Y. Xie, "An invitation control policy for proactive service systems: Balancing efficiency, value, and service level," *Manufacturing & Service Operations Management*, vol. 23, no. 5, pp. 1077–1095, 2021.
- [27] S. Gallino, N. Karacaoglu, and A. Moreno, "Need for speed: The impact of in-process delays on customer behavior in online retail," *Operations Research*, vol. 71, no. 3, pp. 876–894, 2023.
- [28] G. Linden, "Make data useful," 2006.
- [29] T. Everts, "The real cost of slow time vs downtime [powerpoint slides]," Presented at CMG Performance & Capacity Conference, and published on Radware Blog, November 2014, available online via SlideShare. [Online]. Available: <https://www.slideshare.net/slideshow/radware-cmg2014-tammyevertsslowtimevsdowntime/41159280>
- [30] V. Pareto, *Cours d'économie politique*. Librairie Droz, 1964, vol. 1.
- [31] L. Kakalejčík, J. Bucko, and M. Vejačka, "Differences in buyer journey between high-and low-value customers of e-commerce business," *Journal of theoretical and applied electronic commerce research*, vol. 14, no. 2, pp. 47–58, 2019.
- [32] PostgreSQL Global Development Group, "Postgresql 15 documentation," PostgreSQL Global Development Group, 2022, version 15.0, Online; accessed 2025-11-07. [Online]. Available: <https://www.postgresql.org/files/documentation/pdf/15/postgresql-15-US.pdf>
- [33] "Mysql," <https://www.mysql.com/>, 2025, the world's most popular open source database.